

BlurTrace

A Traceable Image-Blurring System — Student Case Study Summary Report

This report summarizes the design, implementation, and problem-solving process behind BlurTrace, a demo system built to explore how sensitive images can be blurred before sharing, while preserving a secure, database-backed way to trace a blurred image back to its original source.

1. Problem Statement

Inappropriate or privacy-violating images are frequently shared on social media platforms without the consent of the people in them, causing real and lasting harm — particularly to children and other vulnerable individuals. This project explores a system that lets a person blur a sensitive image before it is shared, while still allowing an authorized process to recover the original image later, for example if a blurred image circulating online needs to be traced back to its source for investigation or verification.

2. Core Concept

Blurring is a one-way, destructive transformation: once pixel information is averaged or reduced, the original cannot be mathematically reconstructed from the blurred result alone. Rather than attempting to reverse a blur, BlurTrace instead pairs the blurred output with its original at the moment of creation, and stores both together in a database, keyed by a hash of the blurred image. Later, a blurred image can be looked up by that same hash to retrieve its paired original.

This is a database lookup/pairing mechanism, not image reconstruction, and it only works for images this system itself produced.

3. System Architecture

Layer	Technology	Role
Backend	Python, FastAPI	Exposes <code>/api/process</code> , <code>/api/store</code> , <code>/api/find</code> endpoints
Database	PostgreSQL (psycopg2)	Single <code>image_pairs</code> table storing original + blurred bytes, keyed by hash
Image processing	OpenCV (opencv-python-headless)	Gaussian blur and pixelation algorithms

Frontend	Plain HTML / CSS / JS	Three pages: Landing, Convert to Blur, Find the Original
----------	-----------------------	--

Deliberate scope decisions

The following limitations were intentional design choices for this case study, not oversights:

- No encryption — original and blurred bytes are stored as plain BYTEA in Postgres.
- No authentication on the store/find endpoints — acceptable for a local, single-user demo.
- Single table design — one image_pairs table rather than normalized tables.
- Exact-match hashing only — no perceptual/fuzzy hashing, discussed further in Section 5.

4. Algorithms Used

Gaussian Blur

Each pixel's new value is computed as a weighted average of its neighbors, with weights following a Gaussian (bell-curve) distribution — nearby pixels contribute more than distant ones. The intensity slider controls the kernel size: a larger kernel averages over a wider neighborhood, producing a stronger blur. This is a standard spatial convolution operation with no randomness involved.

Pixelation

The image is downsampled to a much lower resolution, then scaled back up using nearest-neighbor interpolation. Each block of the enlarged image becomes a single flat color, producing the blocky mosaic effect. The intensity slider controls how small the downscale target is — a smaller target produces chunkier blocks.

Both transformations are lossy and one-directional: information is genuinely discarded, so there is no mathematical inverse. This is the foundation the entire recovery mechanism is built around.

SHA-256 Hashing (Pixel-Based)

SHA-256 produces a fixed-length, one-way fingerprint of input data: identical input always produces identical output, and any change in the input produces a completely different hash. BlurTrace uses this hash as the database lookup key for a blurred image. Section 6 describes an important refinement made to how this hash is computed during development.

5. The Recovery Mechanism

The 'recovery' process is not reconstruction — it is a keyed database lookup:

- At blur-time: the system computes a hash of the blurred output (Y) and stores the row (original X, blurred Y, hash of Y) together.
- At find-time: an uploaded blurred image is hashed the same way, and the system looks up a row with a matching hash. A match returns X; no match returns a clear 'not found' result.

Because the hash is an exact-match function, this only works reliably for blurred images that BlurTrace itself produced — it is a closed-loop system by design, not a general 'un-blur any image' tool.

6. Problem-Solving Journey

Several concrete issues were found and resolved during development. Each is documented here as evidence of the debugging and design-refinement process behind the final system.

Issue 1 — .env file not loading (Windows / VS Code)

Database connections initially failed with connection errors. The root cause was that db.py relied on the current working directory to locate the .env file, which is unreliable depending on how a process is launched from VS Code on Windows. The fix was to resolve the .env path explicitly, relative to db.py's own file location, and to add clear startup diagnostics (path checked, whether the file exists, whether values loaded) so any future misconfiguration is immediately visible rather than surfacing as a generic error.

Issue 2 — module import mismatch during first run

The FastAPI server failed to start with `ModuleNotFoundError: No module named 'algorithms'`. The cause was a simple filename mismatch — the algorithms file had been saved as `algorithm.py` (singular) rather than `algorithms.py` (plural), which `main.py` imports from. Renaming the file resolved this immediately.

Issue 3 — multipart upload size limit

Once real (larger) images were tested through `/api/store`, requests began failing with 'Part exceeded maximum size of 1024KB.' This is a default safety limit in FastAPI's underlying multipart form parser, applied per form field, that is not exposed through the framework's usual `File()/Form()` parameter declarations. The fix was to parse incoming multipart form data manually in each endpoint, explicitly raising the per-part size limit to accommodate realistic photo sizes. One trade-off: this removed FastAPI's automatically generated upload widgets in its interactive `/docs` page, since those widgets depend on the declarative parameter style. Testing moved to the real frontend instead, which better matched the actual user flow regardless.

Issue 4 — clipboard Copy → Find mismatch (most significant finding)

During edge-case testing, images blurred and stored via the **Copy to Clipboard** button could not later be found via Find the Original — even though Save (direct file download) worked perfectly for the same image. Investigation showed that pasting an image from the OS clipboard causes it to be re-encoded, often with different PNG compression settings or through an intermediate bitmap representation. This produces different file bytes for visually identical pixels — exactly the kind of case the project's original scope notes had already anticipated: exact-byte hashing does not survive re-encoding.

This was confirmed experimentally: re-encoding the same blurred image with a different PNG compression level changed the file size from 32,030 to 22,597 bytes for identical pixel content, and produced a different SHA-256 hash under the original byte-based approach.

Resolution: pixel-based hashing

The hash function was changed to hash the image's decoded pixel array (with its dimensions as a prefix) instead of its raw file bytes. This keeps the system's exact-match design intact — it is still exact matching, just exact on pixel content rather than file encoding — so any lossless re-encoding (such as a clipboard round-trip) no longer breaks the match. The change was isolated to the hashing step alone; no changes were needed to the database schema, endpoints, or any of the project's other scope decisions.

Because this changed what the stored hash represents, previously stored test rows were cleared from the database after the fix, since they had been hashed under the old scheme.

7. Testing Summary

Test	Result
Gaussian blur / pixelation across intensity levels 1–10	Passed — visually verified
Hash determinism (same input -> same hash every time)	Passed
Convert -> Save -> Find (exact file, no re-encoding)	Passed
Convert -> Copy -> paste -> Find (re-encoded via clipboard)	Passed after pixel-hash fix
Find with an unrelated, never-blurred image	Passed — correct 'not found' message
Duplicate Save/Copy of the same blurred result	Passed — reports 'already stored'
Switching between multiple uploaded files mid-session	Passed — workspace resets correctly

8. Known Limitations

- Pixel-based hashing still only matches lossless re-encodings. A lossy re-compression (e.g. saving as a low-quality JPEG) would change pixel values slightly and still fail to match — true perceptual/fuzzy matching remains outside this project's scope.

- No authentication on any endpoint — anyone running the app locally can store or look up any pair. Acceptable for a local, single-user demo only.
- No encryption at rest — both original and blurred images are stored as plain bytes.
- The system only recognizes blurred images it produced itself; it cannot trace images blurred by other tools or platforms.

9. Conclusion

BlurTrace demonstrates that a simple, well-scoped combination of two long-established techniques — destructive image blurring and cryptographic hashing — can be composed into a practical pairing-and-recovery workflow, without requiring any novel reconstruction algorithm. The project's most valuable lesson emerged from real testing rather than design alone: the discovery that exact-byte hashing breaks under common, everyday re-encoding (such as clipboard use) directly validated the project's own stated scope limitation, and led to a targeted, well-reasoned fix — hashing pixel content rather than file bytes — that preserved the system's exact-match design while meaningfully improving its real-world usability.